ARCHITECTURE DASHBOARD

PI0.5 Stage 0: Actual Model, Training Pipeline, and Runtime Wiring

This page is grounded only in the files and artifacts in this workspace: the Stage 0 dataset builder, OpenPI JAX config wrapper, normalization job, feasibility bridge, AlpaSim external driver, and the saved rerun/ablation artifacts. It does not invent a custom reward or a custom loss if none was actually implemented.

Base Model

PI0.5

`gemma_2b_lora` vision-language backbone + `gemma_300m_lora` action expert

Training Mode

LoRA

OpenPI JAX train path, supervised fine-tune, no custom RL reward file in this workspace

Action Shape

50 x 32

Only dims 0-2 are semantically active: `delta_s`, `delta_yaw`, `target_speed`

Best Full-Input Run

A

3 live cameras, 101 policy calls, collision-free rollout completion

Source of Truth

AREA	ACTUAL FILE USED
Constants / contracts	contracts.py
Dataset conversion	build_stage0_dataset.py
Scene selection	select_stage0_candidates.py
OpenPI config wrapper	openpi_stage0.py
Training launcher	train_stage0.py
Normalization	compute_stage0_norm_stats.py
Feasibility bridge	bridge.py
AlpaSim driver	pi05_stage0_model.py
Eval driver config	stage0_pi05_A.yaml
Condition A metrics	RERUN_SUMMARY.txt

What is custom here
Dataset conversion, Stage 0 observation repacking, prompt/state/route wiring, local normalization stats, trajectory bridge and clamps, AlpaSim external-driver integration, per-call trace/dump logging.

What is inherited
The core PI0.5 model family and the OpenPI JAX training script. This workspace wraps OpenPI's train path; it does not implement a new foundation-model backbone from scratch.

What is not present
No custom reward-learning loop, no PPO-style RL file, and no custom objective override were found in the Stage 0 project files. This experiment was a supervised LoRA fine-tune plus inference-time feasibility filtering.

Core Model Configuration Actually Used

FIELD	ACTUAL VALUE

Model family	`pi0.5`
Base weights	`gs://openpi-assets/checkpoints/pi05_base/params`
PaliGemma variant	`gemma_2b_lora`
Action expert variant	`gemma_300m_lora`
Action horizon	`50`
Action dim	`32`
Token limit	`1024`
Freeze strategy	OpenPI `get_freeze_filter()` from the PI0.5 config wrapper
EMA	None
Weights fine-tuned	LoRA adapters on the configured PI0.5 stack, not a full from-scratch model

```
Source: openpi_stage0.make_stage0_train_config(...)

model = Pi0Config(
  pi05=True,
  paligemma_variant="gemma_2b_lora",
  action_expert_variant="gemma_300m_lora",
  action_dim=32,
  action_horizon=50,
  max_token_len=1024,
)

weight_loader = CheckpointWeightLoader(
  "gs://openpi-assets/checkpoints/pi05_base/params"
)
```

What the Model Sees and Predicts

Visual Inputs

- `camera_front_wide_120fov`
- `camera_cross_left_120fov`
- `camera_cross_right_120fov`

- Driver remaps them into OpenPI image slots: `base_0_rgb`, `left_wrist_0_rgb`, `right_wrist_0_rgb`

Structured Inputs

- `1.0 s` ego history at `10 Hz`
- Per-step features: speed, yaw rate, acceleration
- State history flattened to `30` dims
- Route corridor: `32 x 2` future ego-frame points
- Prompt token: `lane_follow`, `left_turn`, or `right_turn`

Action Semantics

- Internal tensor preserved at `50 x 32`
- Only active dims: `0 = delta_s`, `1 = delta_yaw`, `2 = target_speed`
- Other 29 dims are present structurally but ignored semantically by this Stage 0 bridge

Training Pipeline Actually Used

1

Scene Selection

The selector validates NuRec labels, requires at least one explicit left turn and one right turn, then picks exactly 5 clips: left, right, and 3 lane-follow clips.

select_stage0_candidates.py

2

Dataset Build

Raw AV cameras and egomotion are aligned to a 10 Hz timeline, RGB frames are decoded, ego kinematics are derived, and route/action targets are computed.

build_stage0_dataset.py

3

Normalization

State and action stats are computed from the Stage 0 dataset itself. The script validates non-finite values, zero std, and bad quantiles before saving assets.

compute_stage0_norm_stats.py

4

OpenPI JAX Fine-Tune

A Stage 0 wrapper constructs the OpenPI training config, repacks observation tensors, exposes only the active action dims on output, and calls the upstream JAX train script.

openpi_stage0.py

5

Runtime Bridge + Driver

At inference time, normalized outputs are denormalized, rolled into a feasible planar trajectory under kinematic limits, then served to AlpaSim via the custom external driver.

bridge.py / pi05_stage0_model.py

Training Mechanics and Data Transforms

MECHANISM	WHAT ACTUALLY HAPPENS
Dataset repo id	`local/stage0_av_driving`
Frames / episodes	545 frames across 5 episodes
Clip duration target	`20.0` s per clip
Sample rate	`10` Hz
Batch size	`8`
Train steps	`2500`
Save interval	`500`

Log interval	`25`
Workers	`0`
Prompt behavior	`prompt_from_task=True` with default model prompt `drive the route`
Experiment name	`pi05_stage0_av`
WandB	`False`

```
Observation repack used in Stage 0

observation.images.front  -> image.front
observation.images.left   -> image.left
observation.images.right  -> image.right
observation.state         -> state
observation.route         -> route
actions                   -> actions
prompt                    -> prompt

DrivingInputs(...)
  state := concat(ego_state, route.flatten())
  image.base_0_rgb := front
  image.left_wrist_0_rgb := left
  image.right_wrist_0_rgb := right

DrivingOutputs(...)
  return actions[:, :3]
```

Loss / Reward / Objective: What Was Actually Used

What we did use

- OpenPI JAX training path
- LoRA fine-tuning on the PI0.5 stack
- Stage 0 data transforms and normalization
- Checkpoint weight initialization from PI0.5 base params

What we did not add

- No RL reward function in this workspace
- No PPO / policy-gradient loop

- No learned simulator reward model
- No custom loss file overriding the upstream OpenPI train objective

What acts like "safety"

- Inference-time feasibility bridge
- Acceleration clamps
- Yaw-rate clamps
- Lateral-acceleration clamps
- These are post-model corrections, not training rewards

Inference-Time Runtime Path

```
PredictionInput from AlpaSim
  -> resolve live / black / override camera frames
  -> build 30-dim state history
  -> build 32x2 route array
  -> map DriveCommand to prompt token
  -> policy.infer(obs)
  -> denormalize actions
  -> rollout_feasible_trajectory(...)
  -> emit trajectory + headings to simulator
  -> log trace.jsonl and optional raw dumps
```

RUNTIME CONFIG	ACTUAL VALUE
Driver host/port	`0.0.0.0:6789`
Device	`cuda`
Context length	`1`
Output frequency	`10 Hz`
Route commands	`use_waypoint_commands: true`
Trajectory optimizer	`disabled`
Camera modes supported	`normal`, `front_only`, `all_black`, `override`

Tracing	`trace.jsonl` + optional per-call dump dir

Feasibility Bridge and Kinematic Limits

LIMIT	ACTUAL VALUE
max_speed_mps	20.0
min_speed_mps	0.0
max_longitudinal_accel_mps2	4.0
min_longitudinal_accel_mps2	-6.0
max_yaw_rate_radps	0.7
max_lateral_accel_mps2	4.5
wheelbase_m	2.9

Why this matters
The model does not directly hand AlpaSim a finished physically feasible car trajectory. The bridge takes the predicted action chunk, denormalizes it, and rolls it forward while enforcing speed, acceleration, yaw-rate, and lateral-acceleration limits.

Implication
Reported closed-loop behavior is the result of `model + bridge + controller`, not raw unconstrained model output alone.

Condition A: Best Full-Input Run

Policy Calls

101

full session

Collision Max

0.0

no collision

Progress Last

0.4772

same-scene route

Distance

56.19 m

traveled in rollout

METRIC	VALUE
scene_id	clipgt-048b974e-1546-488a-b8f9-d32bff77f5aa
rollout_uuid	5ccdd896-2c06-11f1-bf44-fb9fdb4933dc
prompts	['lane_follow', 'right_turn']
latency_ms_mean_warm	239.208
policy_infer_ms_mean_warm	82.512
any_clamp_calls	76
accel_clamps_total	559
target_speed_first_mean	5.823222
delta_s_first_mean	0.534686
delta_yaw_first_mean	0.001434

camera_front nonzero

101 / 101

camera_left nonzero

101 / 101

camera_right nonzero

101 / 101

warm infer vs 100ms loop

82.5 ms

warm end-to-end vs 100ms loop

239.2 ms

What This Architecture Section Can and Cannot Claim

Supported by the real files
PI0.5 LoRA fine-tuning, 3-camera + state + route conditioning, 50x32 action shape preservation, Stage 0 data conversion, local normalization, feasibility bridge, and AlpaSim external-driver execution.

Supported but only at wrapper level
The backbone internals are only observable here through the actual config fields used: `gemma_2b_lora` and `gemma_300m_lora`. This dashboard does not invent deeper block counts that are not defined in this workspace.

Not supported here
No claim of a custom reward, a new RL algorithm, or a custom training objective beyond the OpenPI train path. If you later inspect the upstream OpenPI repo, that would be a separate deeper backbone report.

Artifact Links

- Condition A rollout video
- Condition A rerun summary
- Condition A trace.jsonl

- Condition A rollout metrics parquet
- Condition A rollout ASL

- Existing compact factual dashboard
- Existing complete Stage 0 HTML report
- Stage 0 full paper draft